

Notes on building a verification layer

What turned out to be harder than it looked

David Condrey · WritersLogic · 31 August 2026

What this is

I built an audio provenance implementation end to end, and these are the things that surprised me while doing it. It is not a specification. Algorithm parameters, key derivations and architecture are out of scope; those are better discussed against running code.

Two of these sections are about mistakes I made. I have left them in because they were the most instructive part and because both were the kind of error that looks fine in review and fails in production.

The first section is about published research rather than my own work, and every claim in it is checkable at its source in under an hour.

1. Nobody has actually measured re-recording

No published audio watermarking method reports a *blind* recovery rate over a real speaker-to-microphone path.

Every physical-path figure in the literature is an oracle measurement. The decoder runs at many candidate time offsets, each result is scored against the known ground-truth payload, and the best score is reported. A deployed detector has no ground truth to score against, so it cannot reproduce those numbers.

I did not expect to find this. I went looking for a method to build on and spent a while assuming I was misreading the papers.

DeAR (AAAI 2023) reports bit accuracy from 99.18% at 5 cm down to 92.68% at 100 cm, playing through a Sennheiser SP10 into an ATR2100-USB microphone. The synchronisation method is stated plainly in the paper: it shifts the captured audio across a range of roughly 113 ms, computes accuracy at each shift, and takes the highest as the final figure. Room type, RT60 and ambient noise are never given. No code or weights were released.

DeepAWR (*Pattern Recognition*, 2025) captures physically too, and its extraction code is public, which is what made this concrete rather than suspected. It sweeps 6000 sample offsets, decodes at each, scores against the known payload, keeps the maximum, and exits early on a perfect hit. It calls a CRC verification function and discards the result rather than using it as the acceptance gate. That last detail is the one I would point anyone at: the honest gate is right there in the code, written and unused.

The methods reporting strong “replay” robustness are not capturing physically at all. AWARE (2026) defines its replay attack as convolution with impulse responses from the Aachen database plus mild noise and a band-pass filter. No loudspeaker, no microphone, no distance. That is a test against the same model of the acoustic path a training distortion layer would use, which is closer to marking your own homework than to a measurement. Even so, roughly one marked clip in nine is missed outright at its lower payload rate. Under those same simulated conditions,

AudioSeal and WavMark both sit at chance.

One result cuts the other way and I think it is the most interesting number in the area. RAW-Bench (Interspeech 2025) retrained existing architectures with reverberation inside the training loop, and lifted a spectral-domain model from 0.45 to 0.95 full-message accuracy under simulated reverb, where a waveform-domain model barely moved. So the property does appear trainable into the right architecture. It is still simulated reverb on a proprietary corpus, which makes it a reason to run the experiment rather than a result to build a claim on.

OPEN Whether public material keeps asserting re-recording survival before a blind physical number exists. A competitor or a technical partner can run the check above in an afternoon.

2. A verdict must not mean more than it can

Four verification responses is the right number. The difficulty is not naming them, it is stopping anything from quietly rounding up into a stronger one.

The trap I care most about is infrastructure. If the registry is unreachable and the system reports “no record found,” a network fault has become a provenance statement about somebody’s work. Those two outcomes have to stay distinct all the way up, which in practice means the type system has to refuse to represent an outage as a result, because relying on every caller to remember will fail eventually.

The second one is subtler and I got it wrong initially. A file that has been through a lossy codec is not the file that was signed, so the hash comparison fails, and the obvious response is to report that the audio changed. But it did not change in any sense the owner would recognise. It was uploaded to a platform. Since most real files travel a lossy path, this is not an edge case, it is the common case, and a system that says “altered” about ordinary distribution is crying wolf on nearly everything it sees.

3. When a soft binding is allowed to speak

This is the hardest question in the system and I do not think there is a comfortable answer.

A hash binding is exact and brittle. A watermark or fingerprint is robust and approximate. Once a file has been transcoded the exact binding necessarily fails, so either the approximate one gets to carry a verdict, or verification only succeeds on untouched files, which is a small fraction of what actually circulates.

I built mine so the soft binding could only ever *downgrade* a result. It would locate the record and then contribute nothing, because a failed hash outranked it absolutely. That felt conservative and safe when I wrote it. It is not conservative, it is useless: the watermark was the component that survived the channel, and I had arranged for its survival to be irrelevant. Every transcoded file came back as though it had been tampered with.

Fixing it introduces the obvious hazard. If an approximate match can produce a successful verification, the acceptance threshold controls how easily anyone can obtain one. What keeps that honest is unglamorous: the threshold gets fixed against a measured false-positive target, chosen before anyone looks at how many real cases pass, and never raised afterwards to rescue a case. A threshold tuned after the fact is indistinguishable from one chosen well, and nobody downstream can tell which they are looking at.

The related rule is precedence. An approximate binding may reasonably stand in for a binding

that is *absent*. Letting it override one that is present and failing means an attacker who damages the exact binding gets the weaker check applied instead.

OPEN Whether an approximate match can ever produce a successful verification, and if so, who owns the threshold and the measurement behind it. That is a governance question more than a technical one.

4. A signature proves possession of a key, not identity

A valid signature shows that whoever produced it held the private key. It says nothing about who they are. Any system that prints a name next to a signature has inserted a trust decision between the two, and the question worth asking is who made it.

The structure I settled on is that a name appears only when a signer credential chains to an anchor the verifier already trusts, and the result names which anchor vouched. Two files can both be “verified” under anchors of wildly different standing, and a consumer deserves to see which.

The corollaries are dull and load-bearing. Revocation has to be checked on the verification path rather than only where it is convenient. Validity windows have to be enforced rather than recorded. Credential chain depth has to be bounded, because it is parsed from untrusted input.

OPEN Who can issue a signer credential, what evidence they must see first, and whether revocation is retroactive. The last has no correct answer, only a documented one.

5. Zero is a bound

A detector that produces no false positives across N trials has not been shown to have a false-positive rate of zero. It has a rate below roughly $3/N$ with reasonable confidence. Several hundred trials supports a claim around one percent, which sounds excellent and is not.

At registry scale one percent means roughly one unregistered file in a hundred could be reported as somebody else’s registered work. These systems fail asymmetrically. A missed detection disappoints an owner; a false attribution is a public statement about a third party that turns out to be wrong.

The corpus matters as much as the count. Figures obtained on synthesised audio describe behaviour on synthesised audio, and real recordings differ in ways that bite: sparse solo instruments, true silence, heavily limited masters. None of them resemble noise, and noise is what most benches are made of, including mine until recently.

6. Two mistakes worth describing

A canonicalisation bug that was actually a signature bug. Signing requires turning a document into bytes the same way every time, and I ported a canonical serialiser from an existing Python implementation. It passed its parity tests. What it did not do was reject integer literals wider than 64 bits, which the JSON parser silently widened into floating point. Two documents that were different to the signer became identical bytes to the verifier. Since those bytes are what gets signed, a signature issued over one document would verify another.

The reason I mention it is the shape of the failure rather than the bug. It lived in a component that looked like formatting and was actually part of the trust boundary, and it was documented

in the code as a known formatting divergence, which is to say somebody had already noticed it and filed it under the wrong category.

A commitment that made the thing it committed to impossible. I wanted a mark that could not be lifted from one recording and pointed at a different record, so I derived the mark's identifier from a digest of the signed record. Clean, and it made substitution detectable.

It is also circular. The record covers the marked audio, so the identifier cannot be computed until after marking, and marking needs the identifier. I had written a specification that could not be implemented, and did not notice until the first end-to-end run came back with the watermark decoding correctly and resolving to nothing.

The interesting part was the fix. Reversing the direction, having the record commit to the identifier instead of the identifier digesting the record, preserved the property and removed the cycle. But it only worked because the identifier also binds the signer's key. Without that, anyone can publish a record claiming your identifier for free, which is a worse problem than the one I started with.

7. Incidental robustness is not adversarial robustness

Robustness testing usually measures survival through codecs, resampling and noise. That is necessary, and it answers a different question from whether the mark survives somebody actively trying to remove it.

An attacker who has read the design knows where the mark lives. Can they estimate and subtract it? Can several files marked with the same key be combined to isolate the carrier? Can a mark be transplanted, or overwritten so the detector returns theirs instead?

What makes such a report meaningful is the perceptual cost of each attack. An attack that removes the mark and ruins the recording is not an attack. The useful form of every row is "removed the mark at a cost of X dB of quality," and the design is defensible when that cost exceeds the value of the theft.

OPEN Whether an adversarial evaluation gets published alongside the robustness figures. Publishing it invites scrutiny, which is rather the point of a system that asks to be checked.

8. What would settle re-recording

One number: a blind, gated detection rate on a physical speaker-to-microphone path, with a false-positive rate measured over unmarked audio at a threshold fixed in advance.

Producing it does not need a training budget. It needs several rooms with measured reverberation times, a few consumer speakers, a few microphones, and a campaign across realistic distances. Two weeks, roughly. That should come before serious model investment, because it is the number that says continue or stop, and because a measured negative result is itself something the field does not currently have.

A realistic target is presence detection, establishing that a recording carries a mark at all, at around a metre in a quiet room. A recoverable payload at conversational distance is materially harder. Anything at venue distance or through a pocket is not supported by evidence I could find.

Worth being clear about what presence is worth even if it works. A short payload recovered through a room identifies a bucket rather than a work, and at registry scale a meaningful fraction

of hits resolve to more than one candidate. That is not a defect to fix later, it is what the channel affords, and a product built on it should promise presence rather than identity.

Sources for section 1: the DeAR paper (AAAI 2023), the public DeepAWR repository, the AWARE paper and repository, and RAW-Bench (Interspeech 2025). Licence positions were checked by reading LICENSE files and model-card metadata directly. Figures from the DeepAWR paper itself were not verified; it is paywalled, so the repository's own result logs were used instead.